

CityTech TTP Summer Bootcamp

Revisiting React

2026-06-29

Agenda

1. Continue: Scattergories
2. Review
3. Follow-Ups: Versioning and Modules
4. Hybrid Apps
5. Ionic

Continue Building: Scattergories

Pick up your **Scattergories** group project where you left off.

- Finish the core, then take on the tiered learning goals
- Full spec: `project_specs/scattergories.md`

Review

What were the important lessons you learned during the Scattergories project?

Follow Up: Prisma 7 Is Brand New

We're using **Prisma 7**, released **late 2025**. It is so new that most resources haven't caught up.

- Online **guides** and **YouTube videos** were mostly written for Prisma 5 or 6
- **Other engineers** may only know the older way of doing things
- **AI assistants** were trained on older versions, so they confidently give outdated advice

Breaking Changes

A **breaking change** is a change in a new version that makes **old code stop working**, or behave differently.

- They cluster around **major version bumps** (like 6 to 7)
- Example: Prisma 7 **removed the Rust query engine**, so some older setup steps no longer apply
- Following an old guide step-by-step can fail because of changes like these

Semantic Versioning (semver)

Version numbers follow **MAJOR.MINOR.PATCH** (for example **7.2.1**):

- **MAJOR** (the **7**): **breaking changes**; old code may stop working
- **MINOR** (the **2**): **new features**, backward-compatible
- **PATCH** (the **1**): **bug fixes**, backward-compatible

So jumping from **6 to 7** signals breaking changes; **7.1** to **7.2** should be safe.

The Manual Is the Source of Truth

When a tool moves fast, the **official docs for your version** are the most reliable source.

- Tutorials, videos, and AI can be **stale**; the docs match the **current release**
- Check the docs **version**, and read the **upgrade / migration guide** for breaking changes
- Prisma docs: <https://www.prisma.io/docs>

Follow Up: ESM vs CommonJS

Two ways JavaScript shares code between files (modules):

- **CommonJS (CJS)**: Node's original system; `require(...)` and `module.exports`
- **ES Modules (ESM)**: the modern standard (browsers and modern Node); `import / export`, enabled with `"type": "module"` in `package.json` (or a `.mjs` file)

```
// CommonJS
const express = require("express");

// ESM
import express from "express";
```

They don't always mix cleanly, so a project picks one (most likely **ESM** going forward).

Which Should You Use?

Use ESM going forward.

- It's the official **ECMAScript** standard
- The **default** in browsers and modern frontend tooling (Vite, React)
- Increasingly the default in **Node**, too

CommonJS is the legacy system: you'll still run into it constantly (recognize `require` / `module.exports`), but **new code should prefer ESM**.

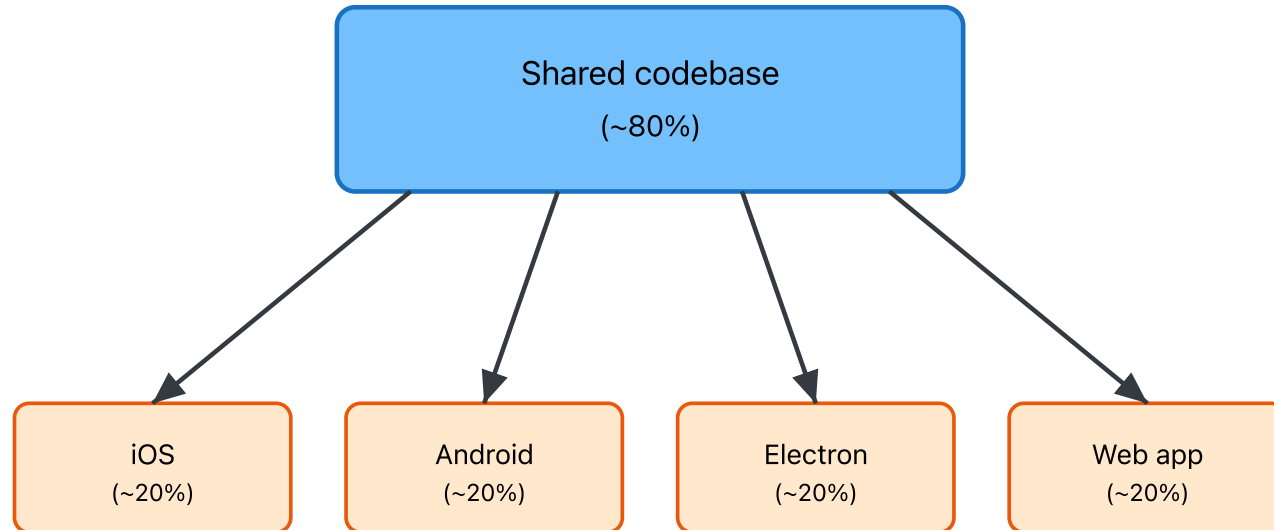
Hybrid Apps

A **hybrid app** shares most of its code across platforms, with a thin platform-specific layer on top.

- Write the bulk **once** and ship to **web, mobile, and desktop**
- Each platform adds a small amount of its own **native glue**

A big reason React skills travel far: the same ideas power web, mobile, and desktop.

One Codebase, Many Platforms



Most of the **code** is shared; each platform adds a thin slice of its own.

Drawbacks of Hybrid

Sharing code has trade-offs:

- **Performance:** usually slower than fully native, especially for graphics- or compute-heavy apps
- **Look and feel:** may not perfectly match each platform's native style
- **Extra layers:** more moving parts to debug, and sometimes a larger app

Go native instead when:

- You're building a performance-critical app
- You need deep or cutting-edge platform features
- One platform's polish matters more than sharing code

Introducing Ionic

Ionic is a framework for building **hybrid apps** with web technology (HTML, CSS, JavaScript).

- Write your app once and ship it to **iOS, Android, and the web**
- Works with **React** (and Angular or Vue)
- Ships a library of **pre-built UI components** that look at home on each platform
- Uses **Capacitor** to reach native device features (camera, storage, and more)

Framework vs Library

- A **library** is code **you call** when you want it; you stay in control and can swap it out
- A **framework** provides the **structure and calls your code** (inversion of control); you build within its rules

ionic is both, and its **UI library is tightly tied to its framework**: to use its components, you adopt ionic's way of doing things, not just drop a few pieces into any project.

Ionic and Capacitor: Official Docs

- **Ionic:** <https://ionicframework.com/docs>
- **Capacitor:** <https://capacitorjs.com/docs>

Using the Ionic Starter Generator

The **Ionic CLI** scaffolds a new app from a starter template.

```
npm install -g @ionic/cli           # install the CLI (once, globally)
ionic start myApp tabs --type=react  # new React app, "tabs" starter
cd myApp
ionic serve                          # run it in the browser
```

- Pick a **starter**: blank, tabs, or sidemenu
- Pick a **framework**: --type=react (or angular, vue)

Ionic Starter Structure

```
myApp/
├── src/                                # your app code
│   ├── main.tsx                       # entry point: mounts the app
│   ├── App.tsx                        # root component + routing
│   ├── pages/                         # your screens (one per tab)
│   ├── components/                   # reusable UI pieces
│   └── theme/variables.css            # Ionic theme (colors, etc.)
├── public/                           # static assets
├── index.html                         # HTML shell
├── ionic.config.json                 # Ionic project config
├── capacitor.config.ts               # native (iOS / Android) config
└── package.json                      # dependencies and scripts
```

Most of your work happens in `src/`, especially `pages/` and `components/`.

Today's Exit Ticket

- ☐ Explain **semver**
- ☐ Explain **ESM vs CommonJS**
- ☐ Explain the **pros and cons of hybrid apps**
- ☐ **Create and run** an Ionic starter